



4 指令系统

计算机在执行的指令长什么样？
程序又是怎样靠这些指令运行起来的？

5 CPU

CPU是怎样一步一步执行指令，并指挥整个计算机工作的？

3 存储器

程序和数据平时放在哪里？
计算机需要时又是怎样找到它们的？

6 总线

计算机各个部件之间是怎样传信息的？

2 数据的表示与运算

数据在计算机里怎样存，计算机怎样拿它们来运算？

7 输入/输出系统

计算机怎样和外部设备连接、交换信息？
用什么方式完成输入输出操作？

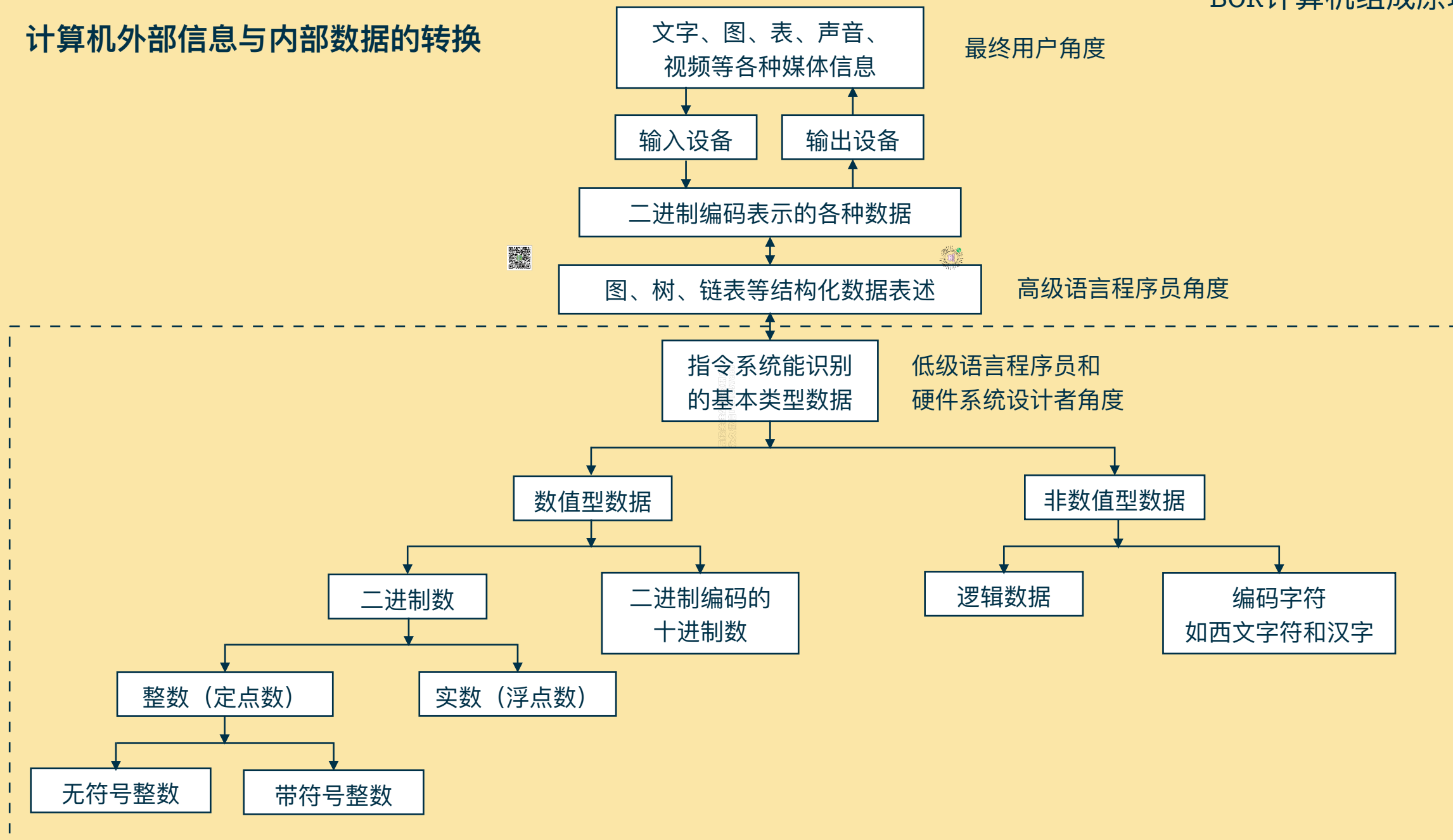
1 概述

计算机系统由什么组成？怎样运行程序？

计算机组成原理



计算机外部信息与内部数据的转换





1. 进制转换

1.1 进位计数制

$$(2585.62)_{10} = 2 \times 10^3 + 5 \times 10^2 + 8 \times 10^1 + 5 \times 10^0 + 6 \times 10^{-1} + 2 \times 10^{-2}$$

$$V(D) = d_n \times 10^n + d_{n-1} \times 10^{n-1} + \dots + d_1 \times 10^1 + d_0 \times 10^0 + d_{-1} \times 10^{-1} + d_{-2} \times 10^{-2} + \dots + d_{-m} \times 10^{-m}$$

d可以是0~9的任意一个，称为基数
 10^i 称为第i位上的权

对于二进制来说
基数为2
第i位上的权是 2^i



1. 进制转换

1.2 十进制与二进制的转换

$$(100101.01)_2 = 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 37.25$$

2^0	1
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024
2^{11}	2048

图灵社区会员 陈文斌 专享 尊重版权

2^{12}	4096
2^{13}	8192
2^{14}	16384
2^{15}	32768
2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625
2^{-5}	0.03125



1. 进制转换

1.3 二进制与十六进制的转换



十六进制：逢十六进一

图灵社区会员专享

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F



1. 进制转换

1.3 二进制与十六进制的转换

整数部分从低位向高位方向每4位用一个等值的十六进制数来替换
最后不足4位时在高位补0凑满4位

小数部分从高位向低位方向每4位用一个等值的十六进制数来替换
最后不足4位时在低位补0凑满4位

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111



1. 进制转换

1.4 十进制与十六进制的转换

$$(2B.5E)_{16} = 2 \times 16^1 + 11 \times 16^0 + 5 \times 16^{-1} + 14 \times 16^{-2} = 43.3671875$$

16^0	1
16^1	16
16^2	256
16^3	4096
16^4	65536
16^5	1048576

图灵社区会员
陈文斌 赠阅

16^{-1}	$\frac{1}{16}$
16^{-2}	$\frac{1}{256}$
16^{-3}	$\frac{1}{4096}$



2. 定点与浮点表示

2.1 定点表示



定点表示法用来对定点小数和定点整数进行表示

对于**定点小数**，其小数点总是固定在数的**最左边**，一般用来表示浮点数的尾数部分

对于**定点整数**，其小数点总是固定在数的**最右边**，用来表示整数



2. 定点与浮点表示

2.2 浮点表示

对于任意一个二进制数 x ，可以表示成如下形式：

$$X = (-1)^s \times M \times R^E$$

数符 s ：取值为0或1，用来决定数 x 的符号

尾数 M ：一个二进制定点小数

阶/指数 E ：一个二进制定点整数

基数 R ：可以取值2，4，16等

任意一个浮点数都可以
用一个定点整数和一个定点小数来表示

基数 R 一定的情况下，尾数 M 的位数反映数 x 的有效位数，它决定了数的表示精度
有效位数越多，表示的精度就越高

指数 E 的位数决定数 x 的表示范围，其值确定了小数点的位置



3. 定点数的编码表示



真值： 一个数的真实大小

机器数： 这个数在计算机里实际存成的二进制编码



3. 定点数的编码表示



无符号数

无符号整数



原码

补码

反码

移码

带符号整数



3. 定点数的编码表示



无符号数

所有二进制位都用来表示数值，没有符号位

$$0 \leq n\text{位无符号数} \leq 2^n - 1$$



3. 定点数的编码表示

3.1 原码表示法



原码

符号位+数值位



当x为正数时: $x_n=0$, $x_i=x_i'$ ($0 \leq i \leq n-1$)

当x为负数时: $x_n=1$, $x_i=x_i'$ ($0 \leq i \leq n-1$)

原码0有两种表示形式:

$[+0]_{\text{原}} = 000\dots0$

$[-0]_{\text{原}} = 100\dots0$



3. 定点数的编码表示

3.1 原码表示法



原码

符号位+数值位



当x为正数时: $x_n=0$, $x_i=x_i'$ ($0 \leq i \leq n-1$)

当x为负数时: $x_n=1$, $x_i=x_i'$ ($0 \leq i \leq n-1$)

原码0有两种表示形式:

$[+0]_{\text{原}} = 000\dots0$

$[-0]_{\text{原}} = 100\dots0$

真值	-10D	-0.101B	AH
原码 (二进制版)	1000 1010B	1101 0000B	0000 1010B
原码 (十六进制版)	8AH	D0H	0AH



3. 定点数的编码表示

3.1 原码表示法



原码

符号位+数值位



原码的加减运算规则

如果是两个异号数相加，则必须判定两个数的绝对值大小
根据判断结果决定结果符号，并用绝对值大的数减去绝对值小的数

现代计算机不用原码表示整数，只用定点原码小数来表示浮点数的尾数部分



3. 定点数的编码表示

3.1 原码表示法



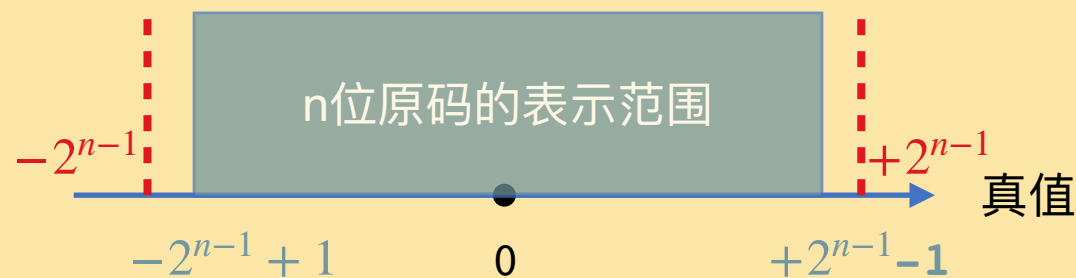
原码

符号位+数值位



原码的表示范围

$$-2^{n-1} + 1 \leq n\text{位原码} \leq 2^{n-1} - 1$$





3. 定点数的编码表示

3.2 反码表示法



反码



反码仅作为原码和补码之间的媒介

对于**正数**：反码与原码相同

对于**负数**：原码的符号位不变，数值位取反，即可得到反码

真值	-10D	-0.101B	AH
原码	1000 1010B	1101 0000B	0000 1010B
反码	1111 0101B	1010 1111B	0000 1010B



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



对于**正数**：补码与原码、反码相同

对于**负数**：原码的符号位不变，数值位取反，末位+1（或，反码末位+1）

真值	-10D	-0.101B	AH
原码	1000 1010B	1101 0000B	0000 1010B
补码	1111 0110B	1011 0000B	0000 1010B



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



补码真理

$$1111\ 0110 = -2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2^1 = -10$$

真值	-10D	-0.101B	AH
补码	1111 0110B	1011 0000B	0000 1010B



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



模运算

数到某个上限以后，不是继续往前排，而是绕回开头重新数



钟表只有12个刻度，但一天有24个小时

当前是10点，再过5个小时是15点（3点）普通加法

: $10+5=15$ ，在钟表上显示不出来

模运算: $(10+5) \bmod 12=3$ ，可以在钟表上显示

x 对 n 取余，就可以让 x 的取值落在 $0 \sim n-1$ 的范围内



补码

补码可以实现加减运算的统一，即用加法实现减法运算



假设现在钟表时针指向11点，要将它拨向6点：

- (1) 倒拨5格： $11-5=6$
- (2) 顺拨7格： $11+7=18 \equiv 6 \pmod{12}$

$$\begin{aligned} 11-5 &\equiv 11+(12-5) \equiv 11+7 \pmod{12} \\ \text{即 } -5 &= 7 \pmod{12} \end{aligned}$$

模12系统里，-5的补码就是7
因为 $-5 \equiv 7 \pmod{12}$ ，所以7可以代替-5



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



假设现在钟表时针指向11点，要将它拨向6点：

(1) 倒拨5格： $11-5=6$

(2) 顺拨7格： $11+7=18 \equiv 6 \pmod{12}$

$$\begin{aligned} 11-5 &\equiv 11+(12-5) \equiv 11+7 \pmod{12} \\ \text{即 } -5 &= 7 \pmod{12} \end{aligned}$$

模12系统里，-5的补码就是7
因为 $-5 \equiv 7 \pmod{12}$ ，所以7可以代替-5

对于某一确定的模，某数A减去小于模的另一数B，可以用A加上-B的补码来代替



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



对于某一确定的模，某数A减去小于模的另一数B，可以用A加上-B的补码来代替

$$[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}}$$

$$[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$[y]_{\text{补}}$ 转换成 $[-y]_{\text{补}}$ ：
全部位取反，末位+1

【例】 $x=1001\ 1100$ ， $y=0100\ 1110$ ，计算 $x-y$ 的真值

计算机中的所有带符号整数都是用补码表示的



3. 定点数的编码表示

3.3 补码表示法

补码

补码可以实现加减运算的统一，即用加法实现减法运算



补码0只有一种表示形式：

$$[0]_{\text{补}} = 000\dots 0$$

而100...0的真值是 -2^{n-1}



3. 定点数的编码表示

3.3 补码表示法

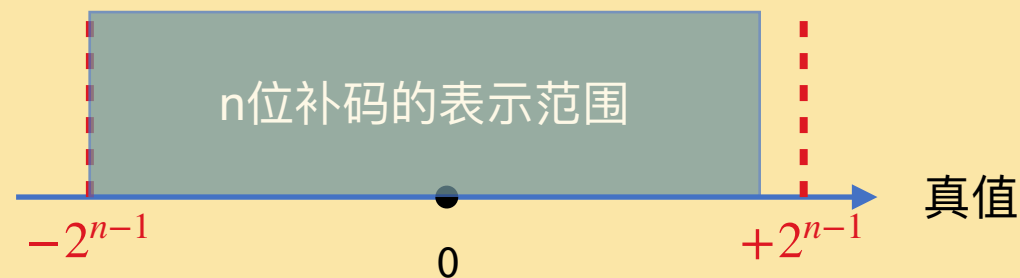
补码

补码可以实现加减运算的统一，即用加法实现减法运算



补码的表示范围

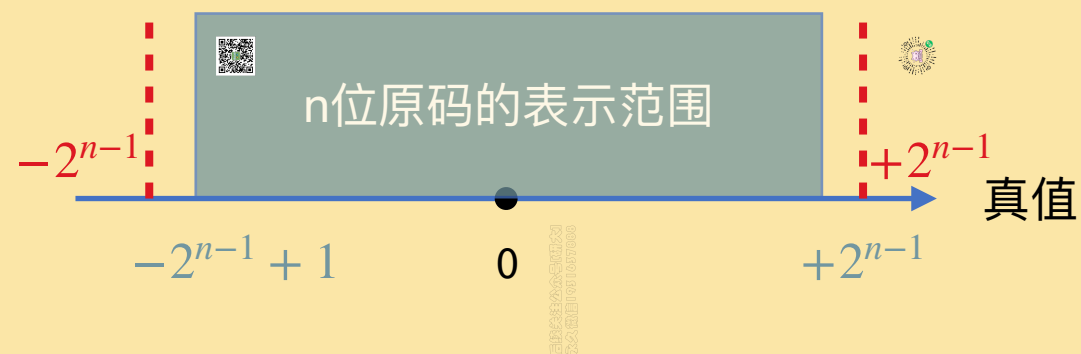
$$-2^{n-1} \leq \text{n位补码} \leq 2^{n-1} - 1$$



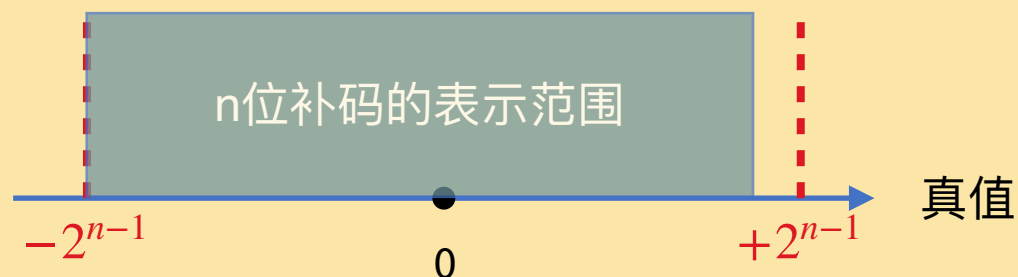


对比发现， n 位补码可以比 n 位原码多表示出一个 -2^{n-1}

$$-2^{n-1} + 1 \leq n\text{位原码} \leq 2^{n-1} - 1$$



$$-2^{n-1} \leq n\text{位补码} \leq 2^{n-1} - 1$$





对比发现， n 位补码可以比 n 位原码多表示出一个 -2^{n-1}



n 位二进制能表示出 2^n 种含义

那么 n 位原码和 n 位补码一定都能表示出 2^n 个数字才对
可是为什么 n 位原码能表示出的数字比 n 位补码少了一个？

原码0有两种表示形式：

$[+0]_{\text{原}} = 000\dots0$

$[-0]_{\text{原}} = 100\dots0$

补码0只有一种表示形式：

$[0]_{\text{补}} = 000\dots0$

而 $100\dots0$ 的真值是 -2^{n-1}

原因在于， $100\dots0$ 这个编码被原码重复用于表示真值0

而 n 位补码拿 $100\dots0$ 这个编码表示出了 -2^{n-1}

所以 n 位补码可以比 n 位原码多表示一个数，这个数就是 -2^{n-1}

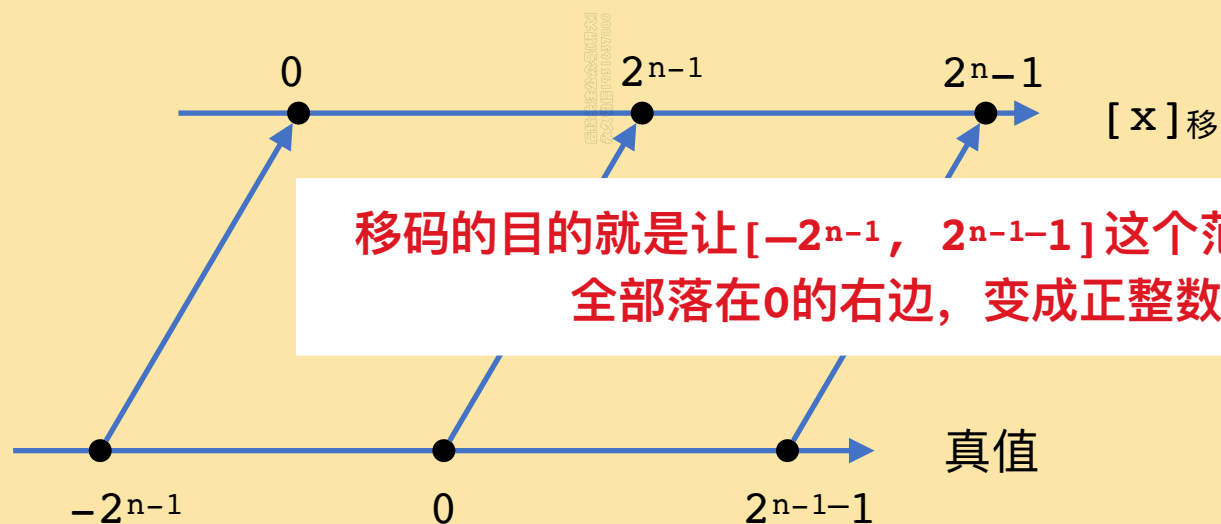


3. 定点数的编码表示

3.4 移码表示法

移码

为了在比较两个不同符号的数时更加直观
可将每个数都加上同一个正数转换为两个正整数再进行比较



注意观察， n 位移码的表示范围和 n 位补码是一样的



3. 定点数的编码表示

3.4 移码表示法

移码

为了在比较两个不同符号的数时更加直观
可将每个数都加上同一个正数转换为两个正整数再进行比较

移码的目的就是让 $[-2^{n-1}, 2^{n-1}-1]$ 这个范围内的数
全部落在0的右边，变成正整数

1001 1110

因为移码一定是正整数

所以可以将其看作无符号数直接翻译

$$1001\ 1110 = 128 + 16 + 8 + 4 + 2 = 158$$

所以移码1001 1110对应的真值为

$$158 - 2^7 = 30$$



3. 定点数的编码表示

3.4 移码表示法

移码

为了在比较两个不同符号的数时更加直观
可将每个数都加上同一个正数转换为两个正整数再进行比较

移码的目的就是让 $[-2^{n-1}, 2^{n-1}-1]$ 这个范围内的数
全部落在0的右边，变成正整数

$[x]_{\text{移}}$ 转换成 $[x]_{\text{补}}$ ：
符号位取反，数值位不变

1001 1110

0001 1110

$16+8+4+2=30$



3. 定点数的编码表示

3.4 移码表示法

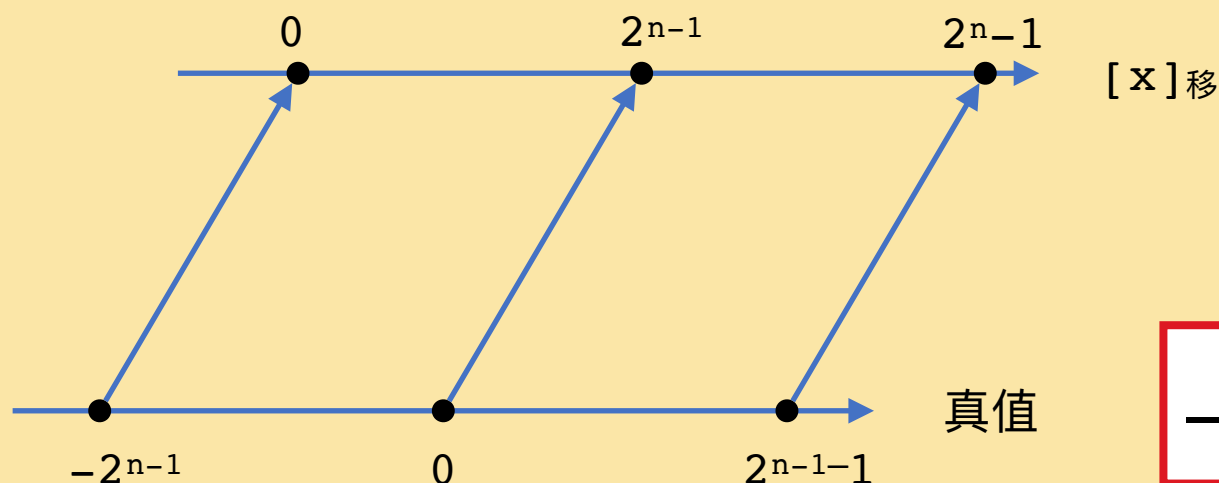
移码

为了在比较两个不同符号的数时更加直观
可将每个数都加上同一个正数转换为两个正整数再进行比较

移码0只有一种表示形式：

$$[0]_{\text{移}} = 100\dots 0$$

移码的表示范围



$$0 \leq \text{移码本身} \leq 2^n - 1$$

$$-2^{n-1} \leq n\text{位移码} \leq 2^{n-1} - 1$$



4. C语言中的类型转换

4.1 C语言中的整数类型



类型	长度
短整型short	16位
整型int	32位
长整型long	32/64位
字符型char	8位



4. C语言中的类型转换

4.2 类型转换



位截断

数据类型转换

位扩展



4. C语言中的类型转换

4.2 类型转换



位截断

长类型转换为短类型时
丢弃高位，保留低位



4. C语言中的类型转换

4.2 类型转换



位截断



长类型转换为短类型时
丢弃高位，保留低位

```
unsigned char x = 173;  
将x强制转换为4位无符号整数
```

1010 1101



4. C语言中的类型转换

4.2 类型转换



位截断



长类型转换为短类型时
丢弃高位，保留低位

```
unsigned char x = 173;  
将x强制转换为4位无符号整数
```

1101

173经过位截断后，结果变成13



4. C语言中的类型转换

4.2 类型转换



位扩展

短类型转换为长类型时
根据数据类型填充高位



4. C语言中的类型转换

4.2 类型转换



位扩展



短类型转换为长类型时
根据数据类型填充高位

零扩展 用于无符号数，高位补0

符号扩展 用于补码表示的有符号数，高位补符号位



4. C语言中的类型转换

4.2 类型转换



位扩展



短类型转换为长类型时
根据数据类型填充高位

将1011扩展至8位

零扩展

用于无符号数，高位补0

符号扩展

用于补码表示的有符号数，高位补符号位

0000 1101

1111 1101

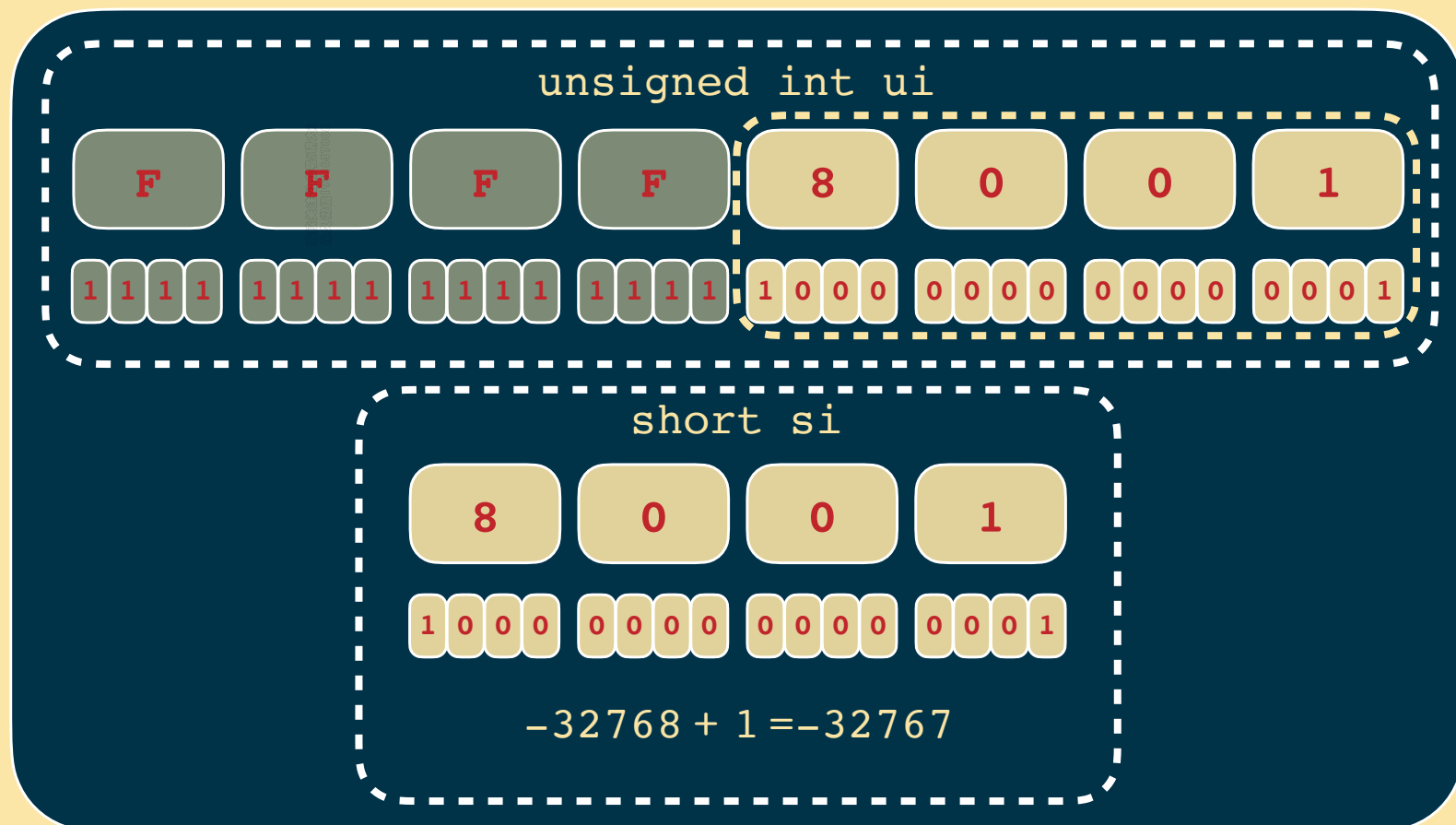


4. C语言中的类型转换

4.2 类型转换

①长类型转短类型：位截断

```
unsigned int ui = 0xFFFF 8001;  
short si = ui;
```





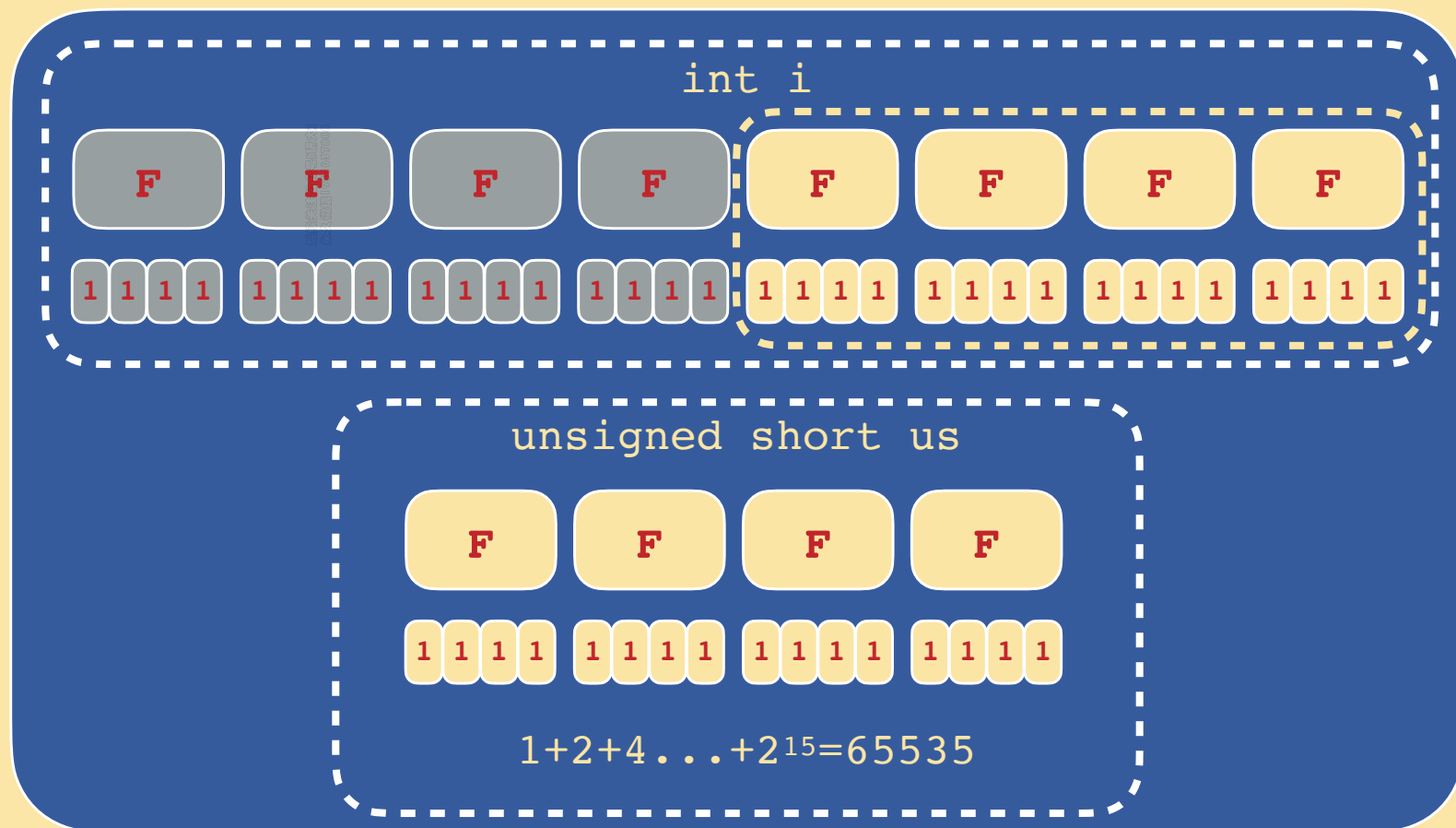
4. C语言中的类型转换

4.2 类型转换

①长类型转短类型：位截断

```
int i = -1;
```

```
unsigned short us = i;
```



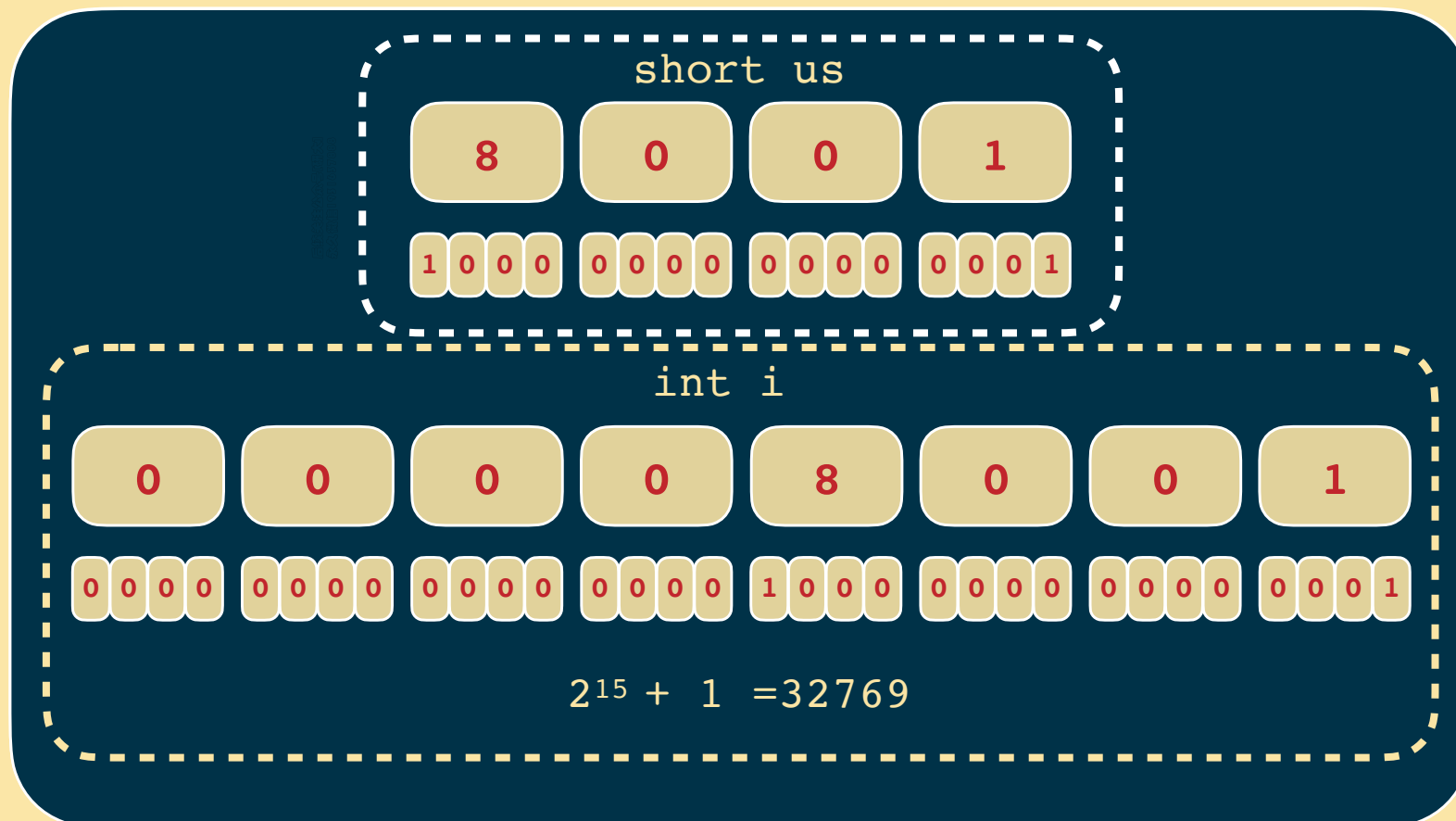


4. C语言中的类型转换

4.2 类型转换

②短类型转长类型：位扩展

```
unsigned short us = 0x8001;  
int i = us;
```



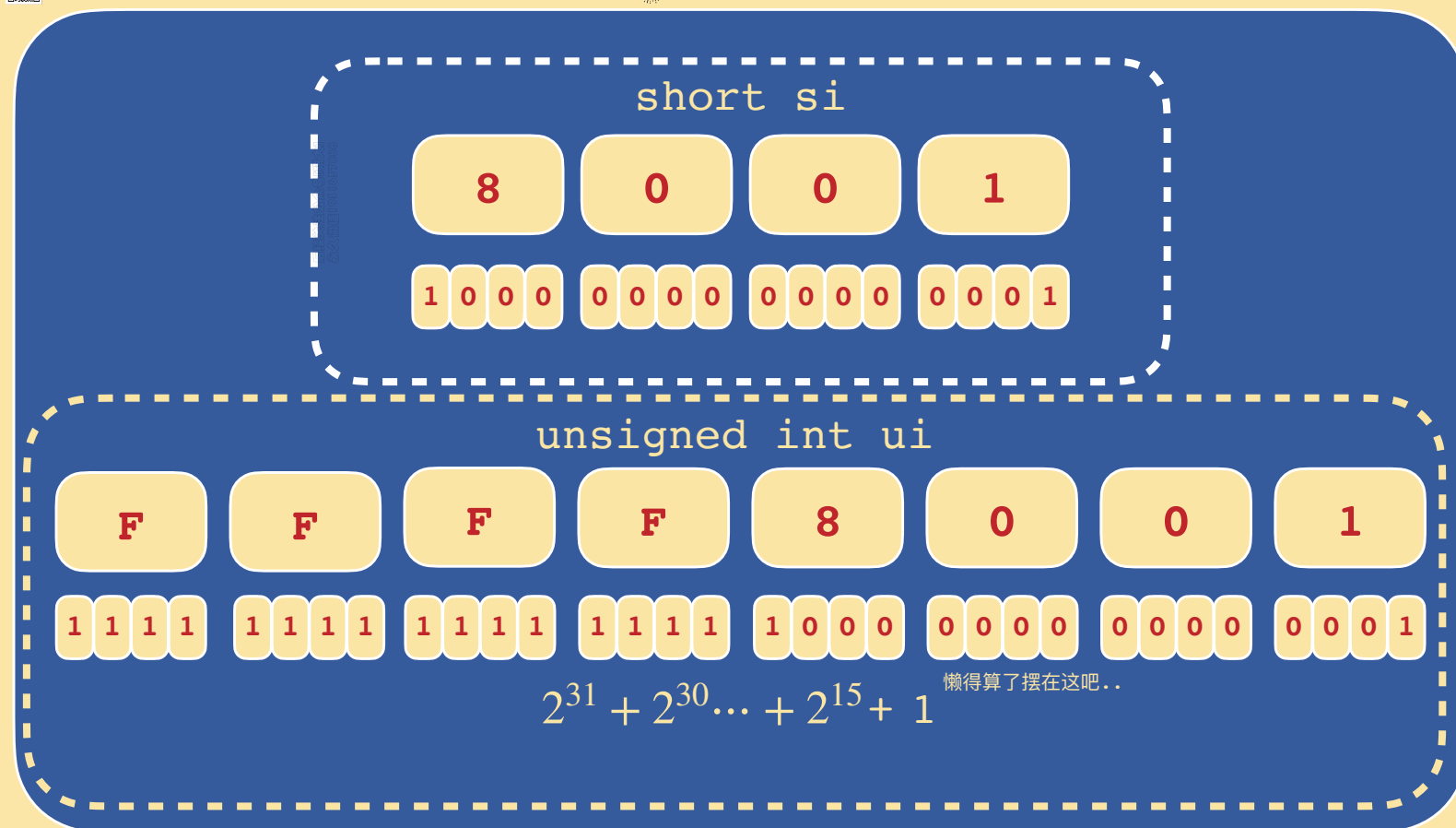


4. C语言中的类型转换

4.2 类型转换

②短类型转长类型：位扩展

```
short si = -32767;  
unsigned int ui = si;
```



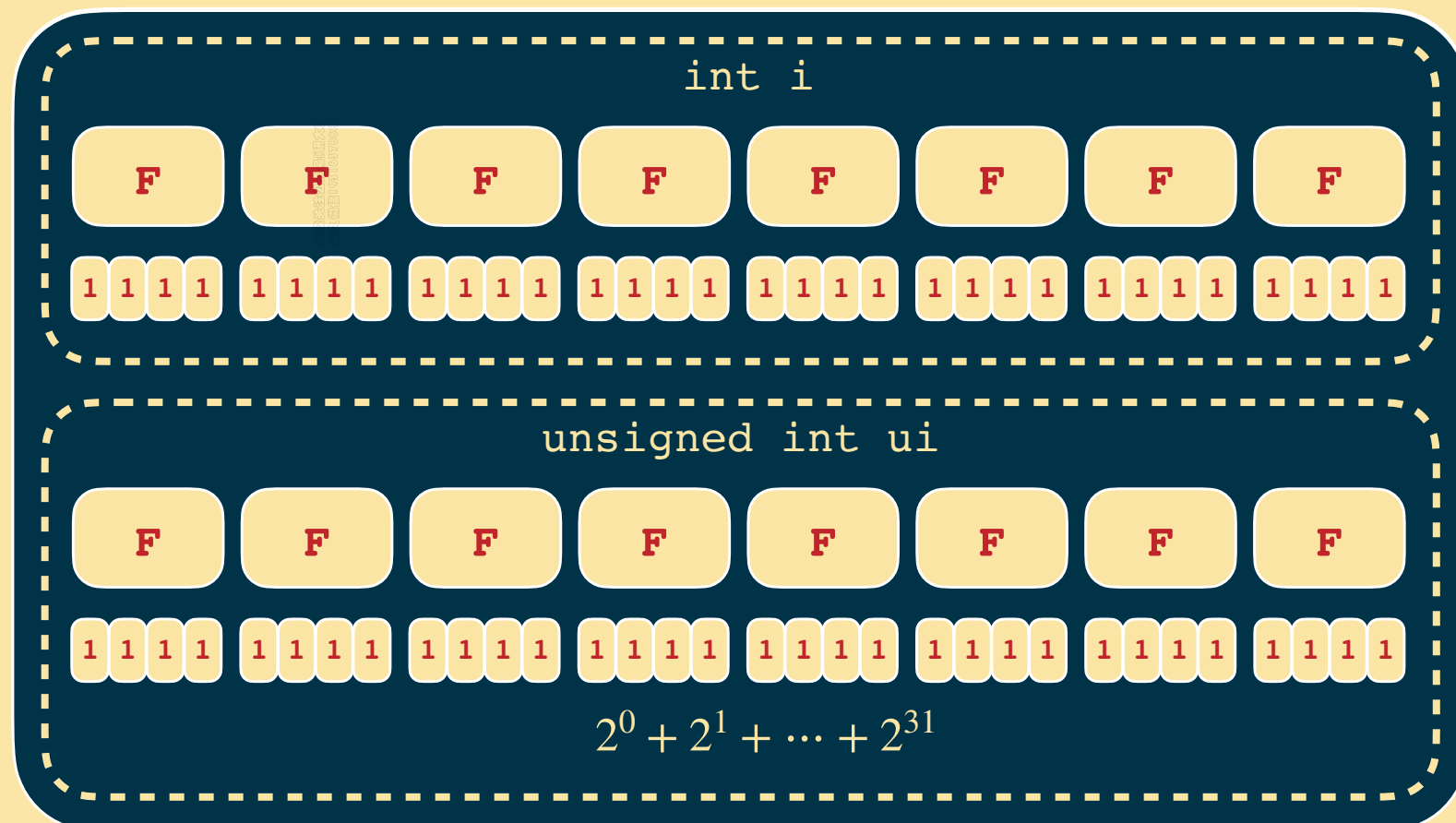


4. C语言中的类型转换

4.2 类型转换

③相同字长转换：仅改变解释方式

```
int i = -1;  
unsigned int ui = i;
```





4. C语言中的类型转换

4.2 类型转换

【真题】在32位计算机上执行下列C语言代码，则ui的真值为（）。

```
short si=-32767;  
unsigned int ui = si;
```

- A. $2^{15}+1$
- B. 2^{15}
- C. $2^{32}-2^{15}-1$
- D. $2^{32}-2^{15}+1$